

HerAI Fellowship

REINFORCEMENT LEARNING

Modul Belajar Praktis dan Mudah Dipahami

Dari agent-environment hingga Q-learning, DQN, dan policy optimization

Level	Intermediate
Estimasi belajar	18-22 jam
Prasyarat	Python dasar, probabilitas, machine learning, dan neural network dasar
Versi	1.0 - 23 Juli 2026

Disusun untuk Participant Module - Project HerAI

Tentang Modul Ini

Modul ini dirancang untuk dibaca berurutan. Setiap bab menggabungkan konsep, analogi, contoh, checkpoint, dan latihan agar peserta tidak hanya menghafal rumus, tetapi memahami alasan sebuah agent mengambil keputusan.

Modul belajar praktis dan mudah dipahami untuk peserta HerAI Fellowship. Materi dimulai dari cara agent berinteraksi dengan environment, lalu berlanjut ke reward, policy, value function, eksplorasi, Q-learning, Deep Q-Network, policy gradient, evaluasi, dan mini project.

Informasi Course

Komponen	Keterangan
Level	Intermediate
Estimasi belajar	18-22 jam
Prasyarat	Python dasar, probabilitas dasar, machine learning, dan pengantar neural network
Bentuk pembelajaran	Materi, contoh, latihan, checkpoint, kuis, diskusi, dan mini project
Hasil akhir	Peserta mampu merancang, melatih, dan mengevaluasi agent RL sederhana secara bertanggung jawab

Capaian Pembelajaran

Setelah menyelesaikan modul ini, peserta diharapkan mampu:

1. Menjelaskan perbedaan reinforcement learning, supervised learning, dan unsupervised learning.
2. Memodelkan masalah sebagai agent, environment, state, action, reward, dan episode.
3. Memahami return, discount factor, policy, value function, Q-value, dan intuisi persamaan Bellman.
4. Menjelaskan trade-off exploration dan exploitation.
5. Menerapkan multi-armed bandit, Monte Carlo, Temporal-Difference, SARSA, dan Q-learning pada kasus sederhana.
6. Memahami alasan penggunaan function approximation dan Deep Q-Network.
7. Mengenali konsep policy gradient, actor-critic, dan PPO pada tingkat pengantar.
8. Mendesain reward dan evaluasi yang tidak mendorong perilaku agent yang merugikan.
9. Membangun mini project Q-learning pada GridWorld dan menyampaikan hasil eksperimennya.

Peta Materi

10. Pengantar Reinforcement Learning
11. Agent, Environment, dan Markov Decision Process

12. Reward, Return, dan Discount Factor
 13. Policy, Value Function, dan Bellman Intuition
 14. Exploration vs Exploitation
 15. Monte Carlo dan Temporal-Difference Learning
 16. SARSA dan Q-Learning
 17. Function Approximation dan Deep Q-Network
 18. Policy Gradient, Actor-Critic, dan PPO
 19. Reward Design, Stabilitas, dan Responsible RL
 20. Evaluasi dan Eksperimen
 21. Mini Project GridWorld
 22. Kuis Akhir, Diskusi, Glosarium, dan Referensi
-

Bab 1 - Pengantar Reinforcement Learning

1.1 Apa itu Reinforcement Learning?

Reinforcement Learning atau RL adalah cara melatih sistem agar dapat mengambil keputusan melalui interaksi. Sistem yang belajar disebut **agent**. Dunia tempat agent bertindak disebut **environment**. Setiap kali agent melakukan suatu **action**, environment memberikan kondisi baru dan sinyal penilaian yang disebut **reward**.

Tujuan agent bukan sekadar mendapatkan reward terbesar pada satu langkah. Tujuan utamanya adalah memilih rangkaian keputusan yang menghasilkan **total reward jangka panjang** sebaik mungkin.

Bayangkan seseorang belajar bermain gim tanpa diberi kunci jawaban untuk setiap gerakan. Ia mencoba tindakan, melihat hasilnya, menerima skor, lalu memperbaiki strategi. Inilah pola dasar RL:

```

agent memilih action
    ↓
environment berubah
    ↓
agent menerima state baru dan reward
    ↓
agent memperbaiki strategi
  
```

1.2 Perbedaan RL dan Supervised Learning

Aspek	Supervised Learning	Reinforcement Learning
Sinyal belajar	Label benar tersedia untuk setiap contoh	Reward dapat tertunda dan tidak selalu menjelaskan tindakan mana yang salah
Data	Dataset relatif tetap	Data dihasilkan dari interaksi agent
Tujuan	Meminimalkan error prediksi	Memaksimalkan total reward jangka panjang
Tantangan utama	Generalisasi ke data baru	Eksplorasi, keputusan berurutan, dan konsekuensi jangka panjang
Contoh	Klasifikasi spam	Agent navigasi, kontrol robot, penjadwalan adaptif

Dalam supervised learning, model diberi contoh input dan jawaban. Dalam RL, agent sering hanya mengetahui apakah hasil akhirnya baik atau buruk. Agent harus mencari tahu sendiri tindakan mana yang berkontribusi terhadap hasil tersebut.

1.3 Kapan RL Cocok Digunakan?

RL paling relevan ketika masalah memiliki beberapa karakteristik berikut:

- Keputusan dilakukan berulang atau berurutan.
- Tindakan saat ini memengaruhi kondisi berikutnya.

- Tidak tersedia label tindakan terbaik untuk setiap situasi.
- Sistem dapat memperoleh feedback berupa reward.
- Ada simulator atau lingkungan aman untuk mencoba banyak tindakan.

Contoh penerapan:

- Pengendalian robot dalam simulasi.
- Optimisasi strategi permainan.
- Pengaturan sumber daya komputasi.
- Penjadwalan produksi atau armada.
- Pengendalian energi pada bangunan.
- Rekomendasi berurutan, dengan perhatian besar terhadap bias dan keselamatan pengguna.

1.4 Kapan RL Tidak Perlu Digunakan?

RL bukan jawaban untuk semua masalah. Jangan memakai RL hanya karena terdengar canggih. Pendekatan lain sering lebih sederhana dan dapat diaudit.

Gunakan alternatif ketika:

- Masalah dapat diselesaikan dengan aturan yang jelas.
- Tersedia dataset berlabel yang kuat dan keputusan tidak berurutan.
- Biaya eksplorasi terlalu berbahaya atau mahal.
- Reward sulit didefinisikan secara bertanggung jawab.
- Simulator tidak mewakili kondisi nyata.
- Sistem harus memiliki jaminan keselamatan yang belum dapat dipenuhi agent.

Prinsip praktis: pilih RL ketika ada keputusan berurutan dan interaksi benar-benar menjadi bagian inti masalah, bukan hanya karena ingin memakai algoritma baru.

Checkpoint Bab 1

23. Mengapa reward berbeda dari label pada supervised learning?
24. Sebutkan satu masalah yang cocok dan satu masalah yang tidak cocok untuk RL.
25. Mengapa proses eksplorasi dapat berbahaya pada sistem dunia nyata?

Latihan Bab 1

Pilih satu skenario berikut: penghemat energi, robot gudang, rekomendasi materi belajar, atau penjadwalan mesin. Tuliskan:

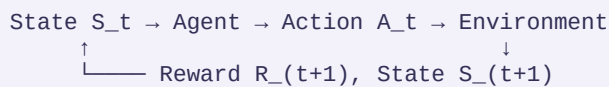
- keputusan apa yang perlu dibuat,
- feedback apa yang mungkin tersedia,
- konsekuensi jangka panjangnya,
- risiko jika agent melakukan eksplorasi tanpa batas.

Bab 2 - Agent, Environment, dan Markov Decision Process

2.1 Siklus Agent-Environment

RL dibangun dari interaksi berulang. Pada langkah waktu t :

26. Agent mengamati state S_t .
27. Agent memilih action A_t .
28. Environment memproses action tersebut.
29. Agent menerima reward R_{t+1} .
30. Agent berpindah ke state S_{t+1} .



Contoh: Robot Pengantar

- **Agent:** perangkat lunak pengendali robot.
- **Environment:** peta gedung, lorong, orang, dan rintangan.
- **State:** posisi robot, arah, jarak ke tujuan, status baterai.
- **Action:** maju, mundur, belok kiri, belok kanan, berhenti.
- **Reward:** positif saat mendekati atau mencapai tujuan; negatif saat menabrak atau boros energi.

2.2 State, Observation, dan Partial Observability

State idealnya berisi informasi yang cukup untuk menentukan konsekuensi tindakan berikutnya. Namun, agent sering hanya menerima **observation**, yaitu sebagian informasi tentang keadaan sebenarnya.

Contoh:

- Kamera robot hanya melihat area di depannya.
- Sistem rekomendasi tidak mengetahui seluruh niat pengguna.
- Sensor dapat memiliki noise atau keterlambatan.

Ketika observation tidak cukup mewakili state, masalah menjadi partially observable. Solusi praktis dapat berupa:

- menyimpan beberapa observation terakhir,
- menambahkan fitur konteks,
- menggunakan model berurutan seperti RNN,
- menggunakan belief state, yaitu perkiraan tentang kondisi sebenarnya.

2.3 Episode dan Continuing Task

Episodic task memiliki awal dan akhir yang jelas. Contoh: satu permainan, satu perjalanan robot, atau satu sesi simulasi.

Continuing task berjalan terus-menerus. Contoh: pengendalian suhu gedung atau pengaturan trafik jaringan.

Pada episodic task, episode biasanya berhenti saat:

- tujuan tercapai,
- agent gagal,
- jumlah langkah maksimum tercapai,
- kondisi keselamatan dilanggar.

Penting membedakan dua jenis akhir:

- **terminated:** episode selesai karena keadaan terminal alami, misalnya tujuan tercapai.
- **truncated:** episode dihentikan oleh batas eksternal, misalnya batas 500 langkah.

Perbedaan ini penting karena target belajar pada kondisi terminal dapat berbeda dari kondisi yang hanya dipotong oleh batas waktu.

2.4 Markov Decision Process

Banyak masalah RL dimodelkan sebagai **Markov Decision Process** atau MDP. Secara ringkas, MDP terdiri dari:

$$\text{MDP} = (S, A, P, R, \gamma)$$

Simbol	Makna
S	himpunan state
A	himpunan action
P	aturan transisi dari state dan action ke state berikutnya
R	aturan reward
gamma	discount factor untuk menimbang masa depan

Markov Property

Sebuah state memenuhi sifat Markov jika informasi yang relevan untuk memprediksi masa depan telah tercakup dalam state saat ini.

Masa depan bergantung pada state sekarang, bukan pada seluruh riwayat secara langsung.

Ini tidak berarti masa lalu tidak penting. Artinya, pengaruh masa lalu telah diringkas ke dalam state sekarang.

Contoh Desain State yang Buruk

Agent harus mengendalikan kendaraan, tetapi state hanya berisi posisi tanpa kecepatan. Dua kendaraan pada posisi yang sama dapat memiliki risiko berbeda jika salah satunya bergerak cepat. Menambahkan kecepatan membuat state lebih informatif.

2.5 Action Space: Discrete dan Continuous

Discrete action space memiliki pilihan terbatas:

{atas, bawah, kiri, kanan}

Continuous action space memiliki nilai tindakan pada rentang tertentu:

sudut kemudi $\in [-30^\circ, 30^\circ]$
 tenaga motor $\in [0, 1]$

Algoritma yang cocok dapat berbeda. Q-learning tabular mudah digunakan pada action diskret kecil, sedangkan kontrol kontinu biasanya membutuhkan pendekatan seperti actor-critic.

Checkpoint Bab 2

31. Apa perbedaan state dan observation?
32. Mengapa kecepatan perlu dimasukkan ke state kendaraan?
33. Apa perbedaan terminated dan truncated?
34. Berikan contoh action space diskret dan kontinu.

Latihan Bab 2 - Memodelkan MDP

Rancang MDP sederhana untuk sistem pengatur lampu lalu lintas. Tentukan:

- state,
 - observation yang tersedia,
 - action,
 - reward,
 - kondisi akhir atau horizon,
 - risiko jika state terlalu sederhana.
-

Bab 3 - Reward, Return, dan Discount Factor

3.1 Reward Bukan Tujuan yang Sempurna

Reward adalah angka yang digunakan untuk mengarahkan proses belajar. Reward bukan selalu representasi sempurna dari tujuan manusia.

Contoh: jika robot hanya diberi reward untuk bergerak cepat, robot dapat memilih jalur berbahaya. Jika sistem belajar hanya diberi reward berdasarkan waktu penggunaan, sistem mungkin mendorong interaksi berlebihan, bukan pembelajaran yang sehat.

Karena itu, reward harus dirancang dengan mempertimbangkan:

- tujuan utama,
- batasan keselamatan,
- dampak jangka panjang,
- kemungkinan agent mencari celah,
- kelompok pengguna yang mungkin dirugikan.

3.2 Return

Reward satu langkah belum cukup untuk menilai strategi. RL menggunakan **return**, yaitu akumulasi reward sejak suatu waktu.

Untuk episode pendek tanpa discount:

$$G_t = R_{(t+1)} + R_{(t+2)} + \dots + R_T$$

Contoh reward sepanjang episode:

```
[-1, -1, -1, +10]
```

Total return:

$$-1 -1 -1 +10 = 7$$

Agent yang mencapai tujuan dengan lebih sedikit langkah memperoleh return lebih tinggi jika setiap langkah diberi biaya kecil.

3.3 Discount Factor

Discount factor γ berada pada rentang 0 sampai 1. Nilai ini menentukan seberapa besar reward masa depan diperhitungkan.

$$G_t = R_{(t+1)} + \gamma R_{(t+2)} + \gamma^2 R_{(t+3)} + \dots$$

Interpretasi sederhana:

Nilai gamma	Perilaku umum
Mendekati 0	agent fokus pada reward yang segera diterima

Nilai gamma	Perilaku umum
Sedang	agent menyeimbangkan hasil dekat dan lebih jauh
Mendekati 1	agent lebih mempertimbangkan konsekuensi jangka panjang

Contoh dengan reward $[0, 0, 10]$:

- Jika $\gamma = 1$, return dari awal adalah 10.
- Jika $\gamma = 0.9$, return dari awal adalah $0 + 0.9(0) + 0.9^2(10) = 8.1$.
- Jika $\gamma = 0.5$, return dari awal adalah 2.5.

Gamma bukan sekadar preferensi moral tentang masa depan. Gamma juga membantu membuat return terbatas pada continuing task dan memengaruhi stabilitas proses belajar.

3.4 Sparse dan Dense Reward

Sparse reward hanya muncul pada kejadian penting, misalnya +1 saat tujuan tercapai. Kelebihannya adalah tujuan lebih jelas, tetapi agent sulit mengetahui langkah perantara yang baik.

Dense reward diberikan lebih sering, misalnya reward kecil saat mendekati tujuan. Ini mempercepat belajar, tetapi berisiko mengubah perilaku agent ke arah yang tidak diinginkan.

Contoh reward shaping:

```
+100 mencapai tujuan
-100 menabrak
-1 setiap langkah
+0.1 jika jarak ke tujuan berkurang
```

Reward shaping harus diuji karena agent dapat mengejar reward perantara tanpa menyelesaikan tugas utama.

3.5 Reward Hacking

Reward hacking terjadi ketika agent mendapatkan reward tinggi dengan cara yang tidak sesuai niat perancang.

Contoh hipotetis:

- Agent pembersih diberi reward berdasarkan jumlah sampah yang dipungut, lalu menjatuhkan dan memungut sampah yang sama berulang kali.
- Agent permainan mendapatkan poin dengan berputar di satu area, bukan menyelesaikan level.
- Sistem rekomendasi mengejar klik, tetapi mengabaikan kualitas atau kesejahteraan pengguna.

Pencegahan praktis:

35. Uji perilaku, bukan hanya angka reward.
36. Tambahkan batasan keselamatan di luar reward.
37. Gunakan beberapa metrik evaluasi.
38. Periksa edge case dan perilaku tak terduga.
39. Libatkan pemangku kepentingan yang terdampak.

Checkpoint Bab 3

40. Apa perbedaan reward dan return?
41. Apa dampak gamma yang sangat kecil?
42. Mengapa dense reward dapat mempercepat belajar sekaligus menimbulkan risiko?
43. Berikan satu contoh reward hacking.

Latihan Bab 3 - Audit Reward

Buat reward awal untuk robot gudang yang harus mengantar barang. Setelah itu, cari minimal tiga kemungkinan perilaku tidak diinginkan dan revisi reward atau batasannya.

Bab 4 - Policy, Value Function, dan Bellman Intuition

4.1 Policy

Policy adalah strategi agent dalam memilih action berdasarkan state.

Policy deterministik:

$$A_t = \pi(S_t)$$

Satu state menghasilkan satu action.

Policy stokastik:

$$\pi(a | s) = \text{probabilitas memilih action } a \text{ pada state } s$$

Contoh policy stokastik pada satu state:

Action	Probabilitas
Atas	0.60
Kanan	0.25
Kiri	0.10
Bawah	0.05

Policy stokastik berguna ketika agent perlu eksplorasi atau ketika beberapa tindakan sama-sama masuk akal.

4.2 State-Value Function

State-value function $V(s)$ mengukur perkiraan return jika agent mulai dari state s dan mengikuti policy tertentu.

$$V_{\pi}(s) = \text{expected return dari state } s \text{ mengikuti policy } \pi$$

Nilai state tinggi berarti state tersebut menjanjikan hasil jangka panjang yang baik menurut policy saat ini.

4.3 Action-Value Function

Action-value function atau **Q-value** $Q(s, a)$ mengukur perkiraan return jika agent:

44. berada pada state s ,
45. memilih action a ,
46. lalu mengikuti policy setelahnya.

$$Q_{\pi}(s, a) = \text{expected return setelah memilih } a \text{ pada } s$$

Jika Q-value diketahui, agent dapat memilih action dengan nilai tertinggi:

```
action terbaik = argmax_a Q(s, a)
```

4.4 Intuisi Bellman

Inti Bellman adalah memecah nilai jangka panjang menjadi dua bagian:

```
nilai sekarang = reward langsung + nilai masa depan yang didiskon
```

Untuk state-value secara intuitif:

```
V(s) = expected [reward berikutnya + gamma × V(state berikutnya)]
```

Untuk Q-value optimal:

```
Q*(s,a) = expected [reward + gamma × max_{a'} Q*(s',a')]
```

Bellman memungkinkan agent memperbarui perkiraan menggunakan satu langkah pengalaman, tanpa harus menunggu semua kemungkinan masa depan benar-benar terjadi.

4.5 Contoh Nilai pada Grid

Misalkan agent berada dua langkah dari tujuan. Reward tujuan adalah **+10**, sedangkan setiap langkah memiliki reward **-1**. Dengan **gamma = 1**:

```
nilai jalur 2 langkah = -1 + 10 = 9
nilai jalur 5 langkah = -1 -1 -1 -1 +10 = 6
```

Agent seharusnya memilih jalur lebih pendek jika tidak ada risiko lain. Jika jalur pendek dekat bahaya, nilai harus mencerminkan kemungkinan gagal.

4.6 Policy Evaluation dan Policy Improvement

Dua ide penting:

- **Policy evaluation:** memperkirakan seberapa baik policy saat ini.
- **Policy improvement:** mengubah policy agar lebih memilih action yang nilainya lebih tinggi.

Keduanya dapat dilakukan berulang:

```
policy → evaluasi nilai → perbaiki policy → evaluasi lagi → ...
```

Pola ini disebut generalized policy iteration dan muncul dalam banyak algoritma RL.

Checkpoint Bab 4

47. Apa perbedaan $V(s)$ dan $Q(s, a)$?
48. Mengapa Q-value berguna untuk memilih action?
49. Jelaskan Bellman dengan bahasa sehari-hari.
50. Apa hubungan policy evaluation dan policy improvement?

Latihan Bab 4

Buat tabel Q sederhana untuk tiga state dan dua action. Isi nilai perkiraan sendiri, lalu tulis policy greedy yang dihasilkan.

Bab 5 - Exploration vs Exploitation

5.1 Dilema Utama

Agent harus memilih antara:

- **Exploitation:** memilih action yang saat ini terlihat paling baik.
- **Exploration:** mencoba action lain untuk memperoleh informasi baru.

Terlalu banyak exploitation membuat agent cepat puas pada strategi yang mungkin belum optimal. Terlalu banyak exploration membuat hasil tidak stabil dan dapat membuang sumber daya.

5.2 Analogi Restoran

Kamu memiliki satu restoran favorit. Exploitation berarti selalu makan di sana karena sudah tahu kualitasnya. Exploration berarti mencoba restoran baru yang mungkin lebih baik, tetapi juga mungkin mengecewakan.

Tujuan bukan memilih salah satu selamanya. Strategi yang baik biasanya banyak mengeksplorasi di awal, kemudian lebih banyak mengeksploitasi ketika pengetahuan meningkat.

5.3 Multi-Armed Bandit

Bandit adalah versi sederhana RL tanpa transisi state yang kompleks. Agent memilih satu dari beberapa action dan menerima reward.

Contoh: tiga variasi judul materi belajar. Setiap variasi memiliki peluang berbeda untuk membuat peserta membuka materi. Sistem perlu belajar variasi yang paling efektif sambil tetap mencoba pilihan lain.

Untuk konteks manusia, eksperimen semacam ini tetap memerlukan batas etika, privasi, dan evaluasi dampak. Reward klik tidak otomatis sama dengan kualitas belajar.

Estimasi Nilai Action

Rata-rata reward action a dapat diperbarui secara incremental:

$$Q_{\text{new}}(a) = Q_{\text{old}}(a) + \alpha \times (\text{reward} - Q_{\text{old}}(a))$$

Jika $\alpha = 1/N(a)$, pembaruan setara dengan rata-rata sampel. Jika lingkungan berubah, α tetap seperti 0.1 dapat membuat estimasi lebih responsif terhadap data terbaru.

5.4 Epsilon-Greedy

Epsilon-greedy adalah strategi sederhana:

- dengan probabilitas $1 - \epsilon$, pilih action terbaik saat ini,
- dengan probabilitas ϵ , pilih action acak.

```
import random

if random.random() < epsilon:
    action = random.choice(actions)      # exploration
else:
    action = max(actions, key=q_value.get) # exploitation
```

Contoh `epsilon = 0.1` berarti sekitar 10% keputusan digunakan untuk eksplorasi.

Epsilon Decay

Epsilon dapat dikurangi perlahan:

```
epsilon = max(epsilon_min, epsilon * decay)
```

Jangan menurunkan epsilon terlalu cepat. Agent dapat berhenti mengeksplorasi sebelum menemukan strategi yang baik.

5.5 Optimistic Initialization dan UCB

Optimistic initialization memberi nilai awal tinggi pada semua action. Agent tertarik mencoba setiap action karena semuanya terlihat menjanjikan pada awalnya.

Upper Confidence Bound atau UCB memilih action berdasarkan kombinasi:

- estimasi reward,
- ketidakpastian karena action jarang dicoba.

Intuisinya:

```
skor action = estimasi hasil + bonus eksplorasi
```

UCB cocok untuk bandit stasioner, tetapi implementasinya berbeda dari epsilon-greedy.

5.6 Contoh Kode Bandit

```
import random

true_rates = [0.15, 0.30, 0.22]
q = [0.0, 0.0, 0.0]
counts = [0, 0, 0]
epsilon = 0.1

for step in range(2000):
    if random.random() < epsilon:
        action = random.randrange(3)
    else:
        action = max(range(3), key=lambda i: q[i])

    reward = 1 if random.random() < true_rates[action] else 0
    counts[action] += 1
    q[action] += (reward - q[action]) / counts[action]

print("Estimasi nilai:", q)
print("Jumlah dipilih:", counts)
```

Kode tersebut mensimulasikan tiga action dengan probabilitas reward berbeda. Hasil setiap run dapat berbeda karena proses acak.

Checkpoint Bab 5

51. Apa risiko exploitation terlalu dini?
52. Apa arti epsilon pada epsilon-greedy?
53. Mengapa epsilon decay perlu dilakukan perlahan?
54. Apa intuisi bonus eksplorasi pada UCB?

Latihan Bab 5

Jalankan simulasi bandit dengan **epsilon** 0, 0.01, 0.1, dan 0.3. Bandingkan:

- total reward,
 - frekuensi action terbaik dipilih,
 - kestabilan hasil antar-run.
-

Bab 6 - Monte Carlo dan Temporal-Difference Learning

6.1 Model-Free Learning

Pada banyak kasus, agent tidak mengetahui aturan transisi environment. Agent hanya memiliki pengalaman:

```
(state, action, reward, next_state)
```

Metode yang belajar langsung dari pengalaman tanpa model transisi lengkap disebut **model-free**.

Dua keluarga dasar:

- Monte Carlo: belajar dari return setelah episode selesai.
- Temporal-Difference: memperbarui nilai sebelum episode selesai menggunakan perkiraan nilai berikutnya.

6.2 Monte Carlo

Monte Carlo menunggu episode selesai, lalu menghitung return yang benar-benar terjadi.

Contoh episode:

```
S0 --A0/-1--> S1 --A1/-1--> S2 --A2/+10--> terminal
```

Return untuk S0 dengan gamma 1 adalah 8. Return untuk S1 adalah 9. Return untuk S2 adalah 10.

Pembaruan nilai:

$$V(S_t) = V(S_t) + \alpha \times (G_t - V(S_t))$$

Kelebihan:

- konsep mudah,
- target berdasarkan hasil episode nyata,
- tidak membutuhkan model environment.

Keterbatasan:

- harus menunggu episode selesai,
- return dapat memiliki variance tinggi,
- kurang cocok untuk continuing task tanpa penyesuaian.

6.3 Temporal-Difference

TD memperbarui nilai setelah satu langkah:

$$V(S_t) = V(S_t) + \alpha \times TD_error$$

Dengan:

$$TD_error = R_{(t+1)} + \gamma V(S_{(t+1)}) - V(S_t)$$

Target TD:

reward langsung + estimasi nilai state berikutnya

TD tidak menunggu episode selesai. Namun, targetnya menggunakan estimasi yang belum tentu akurat. Proses menggunakan estimasi untuk memperbaiki estimasi disebut **bootstrapping**.

6.4 Perbandingan Monte Carlo dan TD

Aspek	Monte Carlo	Temporal-Difference
Waktu update	Setelah episode selesai	Setiap langkah
Target	Return episode	Reward + estimasi nilai berikutnya
Bootstrapping	Tidak	Ya
Variance	Cenderung lebih tinggi	Cenderung lebih rendah
Bias target	Lebih rendah jika episode representatif	Dapat memiliki bias dari estimasi
Continuing task	Kurang langsung	Lebih mudah diterapkan

6.5 Learning Rate

Learning rate **alpha** menentukan besar perubahan nilai.

- Alpha terlalu kecil: belajar lambat.
- Alpha terlalu besar: nilai dapat berosilasi atau tidak stabil.
- Pada environment non-stasioner, alpha tetap membantu agent beradaptasi.

Pembaruan umum:

$$\text{estimasi baru} = \text{estimasi lama} + \alpha \times \text{error}$$

6.6 Bias-Variance Intuition

Monte Carlo memakai hasil episode lengkap. Targetnya lebih dekat pada outcome nyata, tetapi outcome dapat sangat bervariasi.

TD memakai satu langkah dan estimasi. Targetnya lebih stabil, tetapi dapat salah karena nilai state berikutnya juga masih dipelajari.

Tidak ada pilihan tunggal yang selalu terbaik. Pertimbangkan:

- panjang episode,
- noise reward,
- kebutuhan update online,
- stabilitas environment,
- biaya mengumpulkan pengalaman.

Checkpoint Bab 6

- 55. Mengapa Monte Carlo perlu menunggu episode selesai?
- 56. Apa yang dimaksud bootstrapping pada TD?
- 57. Tuliskan bentuk TD error.
- 58. Apa dampak learning rate terlalu besar?

Latihan Bab 6

Gunakan episode reward $[-1, -1, 5]$ dengan $\gamma = 0.9$.

- 59. Hitung return Monte Carlo dari setiap langkah.
 - 60. Misalkan $V(S_t)=1.0$ dan $V(S_{t+1})=2.0$. Hitung TD target dan TD error untuk reward -1 .
-

Bab 7 - SARSA dan Q-Learning

7.1 Control Problem

Policy evaluation hanya menilai policy. Pada control problem, agent ingin menemukan policy yang semakin baik.

SARSA dan Q-learning mempelajari action-value $Q(s, a)$, lalu menggunakan nilai itu untuk memilih action.

7.2 SARSA

Nama SARSA berasal dari urutan:

State, Action, Reward, next State, next Action

Update SARSA:

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha \times [R_{(t+1)} + \gamma Q(S_{(t+1)}, A_{(t+1)}) - Q(S_t, A_t)]$$

SARSA memperbarui nilai berdasarkan action berikutnya yang benar-benar dipilih oleh policy saat ini. Karena itu, SARSA disebut **on-policy**.

Jika policy masih banyak mengeksplorasi, nilai SARSA ikut mempertimbangkan risiko dari tindakan eksploratif.

7.3 Q-Learning

Update Q-learning:

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha \times [R_{(t+1)} + \gamma \max_a Q(S_{(t+1)}, a) - Q(S_t, A_t)]$$

Q-learning menggunakan action terbaik menurut estimasi pada state berikutnya, walaupun agent mungkin memilih action lain untuk eksplorasi. Karena itu, Q-learning disebut **off-policy**.

7.4 SARSA vs Q-Learning

Aspek	SARSA	Q-Learning
Tipe	On-policy	Off-policy
Target berikutnya	Q dari action yang benar-benar dipilih	Q maksimum dari semua action
Memperhitungkan eksplorasi dalam target	Ya	Tidak secara langsung
Perilaku umum	Dapat lebih konservatif saat eksplorasi berisiko	Mengejar policy greedy optimal berdasarkan estimasi

Ilustrasi Jalur Berbahaya

Bayangkan jalur tercepat berada di dekat tebing. Dengan epsilon-greedy, agent kadang mengambil action acak.

- Q-learning dapat menilai jalur tepi tebing sebagai terbaik karena targetnya selalu memakai action maksimum berikutnya.
- SARSA dapat memilih jalur lebih aman karena targetnya mempertimbangkan action aktual, termasuk kemungkinan eksplorasi.

Hasil bergantung pada environment, epsilon, dan hyperparameter. Ini adalah intuisi, bukan jaminan universal.

7.5 Q-Table

Untuk state dan action diskret kecil, Q-value dapat disimpan dalam tabel.

State	Atas	Bawah	Kiri	Kanan
S0	0.4	-0.2	0.1	0.8
S1	0.7	0.0	0.3	0.2
S2	-0.5	0.9	0.4	0.1

Policy greedy memilih nilai terbesar pada setiap baris.

7.6 Pseudocode Q-Learning

```
inisialisasi Q(s,a)
untuk setiap episode:
    reset environment, dapatkan state
    selama episode belum selesai:
        pilih action dengan epsilon-greedy
        jalankan action
        terima reward dan next_state
        target = reward + gamma * max Q(next_state, semua_action)
        Q(state, action) += alpha * (target - Q(state, action))
        state = next_state
```

Pada state terminal, nilai masa depan biasanya dianggap nol:

```
target terminal = reward
```

7.7 Implementasi Ringkas

```
import random
from collections import defaultdict

Q = defaultdict(lambda: [0.0, 0.0, 0.0, 0.0])
alpha = 0.1
gamma = 0.99
epsilon = 0.2

for episode in range(1000):
    state, info = env.reset()
    done = False

    while not done:
        if random.random() < epsilon:
```

```

        action = env.action_space.sample()
    else:
        action = max(range(4), key=lambda a: Q[state][a])

    next_state, reward, terminated, truncated, info = env.step(action)
    done = terminated or truncated

    future = 0.0 if terminated else max(Q[next_state])
    target = reward + gamma * future
    Q[state][action] += alpha * (target - Q[state][action])
    state = next_state

```

Kode di atas adalah pola umum. Bentuk state harus dapat digunakan sebagai key dictionary. Pada environment nyata, perhatikan perbedaan terminal dan time limit dengan lebih teliti.

7.8 Hyperparameter Penting

- **alpha**: kecepatan pembaruan.
- **gamma**: bobot masa depan.
- **epsilon**: tingkat eksplorasi.
- jumlah episode: banyaknya pengalaman.
- epsilon decay: perubahan eksplorasi sepanjang training.
- max steps: batas panjang episode.

Jangan menilai algoritma hanya dari satu kombinasi hyperparameter atau satu random seed.

Checkpoint Bab 7

61. Apa arti on-policy dan off-policy?
62. Apa perbedaan target SARSA dan Q-learning?
63. Mengapa Q-table tidak cocok untuk state berukuran sangat besar?
64. Kapan nilai masa depan dibuat nol?

Latihan Bab 7

Buat satu contoh transisi:

```
Q(s,a)=2.0, reward=1, gamma=0.9, max Q(s',.)=4.0, alpha=0.1
```

Hitung:

65. target Q-learning,
66. TD error,
67. Q-value baru.

Bab 8 - Function Approximation dan Deep Q-Network

8.1 Mengapa Q-Table Tidak Cukup?

Q-table bekerja baik ketika jumlah state-action kecil. Namun, banyak masalah memiliki:

- state kontinu,
- gambar berukuran besar,
- kombinasi state yang sangat banyak,
- observation yang berubah terus-menerus.

Misalnya, gambar 84 x 84 piksel memiliki ruang kemungkinan sangat besar. Menyimpan satu baris Q untuk setiap gambar tidak realistis.

Solusinya adalah **function approximation**:

$$Q(s, a; \theta)$$

Model berparameter θ menerima state dan menghasilkan perkiraan Q-value.

8.2 Linear Function Approximation

Model linear dapat menggunakan fitur state:

$$Q(s, a; \theta) = \theta_a^T \phi(s)$$

$\phi(s)$ adalah representasi fitur state. Model linear lebih efisien daripada tabel, tetapi kemampuannya terbatas untuk pola non-linear.

8.3 Deep Q-Network

Deep Q-Network atau DQN memakai neural network untuk memperkirakan Q-value.

Input:

state atau observation

Output untuk action diskret:

$$[Q(s, a_1), Q(s, a_2), \dots, Q(s, a_k)]$$

Agent memilih action dengan Q-value terbesar, sambil tetap melakukan eksplorasi selama training.

8.4 Masalah Target yang Bergerak

Jika network yang sama digunakan untuk:

- menghasilkan prediksi saat ini,
- sekaligus menghasilkan target,

maka target berubah setiap kali network diperbarui. Ini dapat membuat training tidak stabil.

DQN menggunakan **target network**, yaitu salinan network yang diperbarui lebih jarang.

```
online network: diperbarui setiap langkah optimisasi
target network: disalin dari online network secara berkala
```

Target DQN:

```
y = reward + gamma × max_a' Q_target(next_state, a')
```

Loss:

```
loss = (y - Q_online(state, action))^2
```

8.5 Experience Replay

Pengalaman disimpan ke replay buffer:

```
(state, action, reward, next_state, done)
```

Training mengambil mini-batch acak dari buffer.

Manfaat:

- pengalaman dapat digunakan ulang,
- mengurangi korelasi antar-sampel berurutan,
- meningkatkan efisiensi data,
- membuat mini-batch lebih beragam.

Namun, distribusi data replay tetap bergantung pada policy pengumpulan pengalaman.

8.6 Alur DQN

1. Agent mengamati state.
2. Agent memilih action dengan epsilon-greedy.
3. Transisi disimpan ke replay buffer.
4. Sampel mini-batch diambil.
5. Target dihitung memakai target network.
6. Online network diperbarui dengan gradient descent.
7. Target network diperbarui berkala.

8.7 Pseudocode Training Step

```
# batch berisi state, action, reward, next_state, terminated
q_all = online_net(state)
q_selected = q_all.gather(1, action)

with no_grad():
    next_q = target_net(next_state).max(dim=1)
    target = reward + gamma * (1 - terminated) * next_q

loss = mean_squared_error(q_selected, target)
optimizer.zero_grad()
loss.backward()
clip_grad_norm_(online_net.parameters(), max_norm=10.0)
```

```
optimizer.step()
```

Pseudocode ini menunjukkan logika umum, bukan program lengkap.

8.8 Tantangan DQN

- Overestimation karena operator maksimum.
- Sensitif terhadap reward scale dan hyperparameter.
- Training dapat tidak stabil.
- Membutuhkan cukup pengalaman.
- Hanya langsung cocok untuk action diskret.
- Nilai loss kecil tidak selalu berarti policy bagus.

Pengembangan yang umum:

- Double DQN untuk mengurangi overestimation.
- Dueling network untuk memisahkan estimasi state value dan advantage.
- Prioritized replay untuk memprioritaskan pengalaman dengan error besar.

Modifikasi tidak otomatis memperbaiki semua kasus. Gunakan baseline dan eksperimen terkontrol.

8.9 Kapan Tidak Memakai DQN?

DQN kurang cocok ketika:

- action kontinu,
- observasi sederhana dan Q-table sudah cukup,
- data sangat mahal dan algoritma tidak efisien sampel,
- reward atau simulator belum tervalidasi,
- keselamatan memerlukan kendali eksplisit.

Checkpoint Bab 8

68. Mengapa Q-table tidak cocok untuk gambar?
69. Apa fungsi target network?
70. Apa manfaat replay buffer?
71. Mengapa loss DQN tidak cukup untuk menilai policy?

Latihan Bab 8

Gambarkan arsitektur DQN untuk observation berukuran 8 fitur dan 4 action. Tentukan:

- ukuran input,
- dua hidden layer,
- ukuran output,
- informasi yang disimpan di replay buffer.

Bab 9 - Policy Gradient, Actor-Critic, dan PPO

9.1 Dari Nilai ke Policy Langsung

Metode value-based seperti Q-learning belajar nilai action, lalu memilih action terbaik. **Policy gradient** mengoptimalkan policy secara langsung.

Policy berparameter:

```
pi_theta(a | s)
```

Neural network menghasilkan distribusi probabilitas action. Pada action kontinu, network dapat menghasilkan parameter distribusi seperti mean dan standard deviation.

9.2 Tujuan Policy Gradient

Tujuannya adalah meningkatkan expected return:

```
J(theta) = expected total return dari policy pi_theta
```

Intuisi update:

- action yang menghasilkan return lebih baik dibuat lebih mungkin,
- action yang menghasilkan return buruk dibuat kurang mungkin.

Bentuk intuitif gradient:

```
gradient  $\approx$  grad log pi_theta(action | state)  $\times$  return
```

9.3 REINFORCE

REINFORCE adalah algoritma policy gradient dasar:

72. Jalankan satu episode dengan policy saat ini.
73. Hitung return dari setiap langkah.
74. Tingkatkan probabilitas action yang diikuti return tinggi.
75. Turunkan probabilitas action yang diikuti return rendah.

Kelemahannya adalah variance tinggi. Dua episode dari state mirip dapat memberi return sangat berbeda.

9.4 Baseline dan Advantage

Untuk mengurangi variance, return dibandingkan dengan baseline, biasanya value function.

```
advantage = return - V(state)
```

Interpretasi:

- advantage positif: action lebih baik dari perkiraan rata-rata.
- advantage negatif: action lebih buruk dari perkiraan rata-rata.

Daripada memperkuat semua action yang memiliki return positif, agent memperkuat action yang lebih baik dari ekspektasi pada state tersebut.

9.5 Actor-Critic

Actor-critic memiliki dua komponen:

- **Actor:** policy yang memilih action.
- **Critic:** memperkirakan value atau advantage untuk menilai action actor.

```
state → actor → action
action + hasil → critic → sinyal perbaikan actor
```

Actor-critic dapat diterapkan pada action diskret maupun kontinu, bergantung pada desain policy.

9.6 PPO pada Tingkat Pengantar

Proximal Policy Optimization atau PPO dirancang agar update policy tidak berubah terlalu ekstrem dalam satu langkah.

Masalahnya: jika policy diubah terlalu besar berdasarkan batch pengalaman terbatas, performa dapat runtuh.

PPO membatasi perubahan menggunakan objective yang di-clip. Intuisi sederhananya:

```
perbaiki policy,
tetapi jangan menjauh terlalu cepat dari policy lama
```

Komponen umum PPO:

- policy lama dan policy baru,
- rasio probabilitas action,
- advantage estimate,
- clipping,
- value loss,
- entropy bonus untuk mendorong eksplorasi.

PPO populer karena relatif stabil dan cukup fleksibel, tetapi tetap membutuhkan tuning, evaluasi, serta data simulasi yang cukup.

9.7 Entropy

Entropy mengukur keragaman distribusi action. Policy dengan entropy tinggi lebih acak. Entropy bonus dapat mencegah policy menjadi deterministik terlalu cepat.

Terlalu banyak entropy membuat agent terus acak. Terlalu sedikit entropy dapat menyebabkan eksplorasi berhenti terlalu awal.

9.8 Value-Based vs Policy-Based

Aspek	Value-Based	Policy-Based
Yang dipelajari	Nilai action atau state	Policy langsung
Contoh	Q-learning, DQN	REINFORCE, PPO

Aspek	Value-Based	Policy-Based
Action kontinu	Tidak langsung cocok untuk DQN	Dapat dimodelkan dengan distribusi kontinu
Stabilitas	Dapat sensitif terhadap bootstrapping	Dapat sensitif terhadap variance dan besar update
Output	Q-value	Probabilitas atau parameter action

Checkpoint Bab 9

76. Apa yang dioptimalkan policy gradient?
77. Apa fungsi baseline?
78. Apa peran actor dan critic?
79. Apa intuisi clipping pada PPO?

Latihan Bab 9

Untuk tiga langkah dengan return $[4, 1, -2]$ dan baseline $[2, 2, 0]$, hitung advantage sederhana. Jelaskan action mana yang seharusnya diperkuat atau dilemahkan.

Bab 10 - Reward Design, Stabilitas, dan Responsible RL

10.1 Optimizing the Metric Is Not the Same as Solving the Problem

Agent mengoptimalkan sinyal yang diberikan. Jika sinyal tidak lengkap, agent dapat menghasilkan perilaku yang secara matematis benar tetapi secara manusia tidak dapat diterima.

Sebelum training, tulis tiga lapisan tujuan:

80. **Objective:** hasil utama yang ingin dicapai.
81. **Constraints:** hal yang tidak boleh dilanggar.
82. **Monitoring metrics:** indikator yang tidak langsung dioptimalkan tetapi harus diawasi.

Contoh pengatur energi:

- Objective: menurunkan konsumsi energi.
- Constraint: suhu ruangan harus tetap dalam rentang nyaman dan aman.
- Monitoring: keluhan pengguna, variasi suhu antar-ruang, frekuensi switching perangkat.

10.2 Reward Scale dan Clipping

Reward yang sangat besar atau memiliki skala tidak konsisten dapat membuat update tidak stabil. Beberapa strategi:

- normalisasi reward,
- reward clipping,
- rescaling,
- memisahkan komponen reward untuk audit.

Namun, clipping dapat menghilangkan perbedaan penting. Misalnya reward 10 dan 100 sama-sama menjadi 1 setelah clipping. Keputusan harus disesuaikan dengan masalah.

10.3 Termination Design

Kondisi akhir memengaruhi apa yang dipelajari agent.

Kesalahan umum:

- agent mendapat reward nol saat melanggar keselamatan, tetapi episode tidak segera dihentikan,
- timeout diperlakukan sama dengan terminal alami,
- agent menemukan cara mengakhiri episode untuk menghindari penalti lebih besar,
- reset environment menghasilkan kondisi yang tidak realistis.

Uji skenario akhir satu per satu.

10.4 Sim-to-Real Gap

Agent yang berhasil di simulator belum tentu berhasil di dunia nyata. Perbedaannya dapat berasal dari:

- sensor lebih bising,
- dinamika fisik tidak akurat,
- perilaku manusia tidak terwakili,

- kondisi langka tidak muncul,
- latensi dan kegagalan perangkat.

Mitigasi:

- domain randomization,
- validasi simulator,
- uji bertahap,
- human oversight,
- fallback policy,
- batas action yang keras,
- monitoring dan kill switch.

10.5 Offline RL dan Data Historis

Pada beberapa kasus, eksplorasi online terlalu berbahaya. Offline RL mencoba belajar dari dataset interaksi yang telah dikumpulkan.

Tantangan utama: agent dapat memilih action yang jarang atau tidak pernah ada dalam data, sehingga estimasi nilainya tidak dapat dipercaya. Karena itu, offline RL bukan sekadar menjalankan Q-learning pada log historis.

Pertanyaan audit:

- Siapa yang menghasilkan data?
- Policy apa yang digunakan saat data dikumpulkan?
- Kelompok mana yang kurang terwakili?
- Apakah reward dapat dihitung dengan benar dari log?
- Apakah ada perubahan kondisi setelah data dikumpulkan?

10.6 Human-in-the-Loop

Untuk sistem berdampak tinggi, manusia dapat dilibatkan pada:

- menyetujui action tertentu,
- meninjau episode gagal,
- memberi feedback,
- menentukan batas keselamatan,
- menghentikan deployment,
- menilai dampak yang tidak tertangkap reward.

Human-in-the-loop bukan alasan untuk mengabaikan desain sistem. Beban manusia, konsistensi keputusan, dan mekanisme eskalasi juga perlu dirancang.

10.7 Checklist Responsible RL

Sebelum eksperimen:

- Apakah RL benar-benar diperlukan?
- Apakah simulator cukup representatif?
- Apakah reward memiliki celah yang jelas?
- Apakah eksplorasi dapat merugikan manusia atau sistem?
- Apakah ada batas action?

Sebelum deployment:

- Apakah performa diuji pada banyak seed dan skenario?

- Apakah metrik keselamatan terpisah dari reward?
- Apakah ada fallback policy?
- Apakah monitoring dapat mendeteksi perubahan distribusi?
- Siapa yang berwenang menghentikan sistem?

Checkpoint Bab 10

83. Mengapa objective, constraints, dan monitoring perlu dipisahkan?
84. Apa itu sim-to-real gap?
85. Mengapa offline RL memiliki risiko out-of-distribution action?
86. Sebutkan dua mekanisme keselamatan di luar reward.

Latihan Bab 10 - Red Team Reward

Ambil reward dari latihan sebelumnya. Bertindaklah sebagai agent yang mencari celah. Tuliskan lima cara memperoleh reward tanpa memenuhi tujuan sebenarnya. Setelah itu, tambahkan batasan dan metrik untuk mengurangi risiko tersebut.

Bab 11 - Evaluasi dan Eksperimen RL

11.1 Mengapa Evaluasi RL Sulit?

Hasil RL dipengaruhi oleh:

- random initialization,
- urutan pengalaman,
- stochastic environment,
- exploration schedule,
- implementasi detail,
- hyperparameter,
- panjang training.

Satu run yang berhasil belum cukup. Model dapat beruntung pada seed tertentu.

11.2 Training Return dan Evaluation Return

Saat training, policy masih melakukan eksplorasi. Saat evaluasi, policy biasanya dijalankan dengan eksplorasi dikurangi atau dimatikan.

Pisahkan:

- **training return:** performa selama pengumpulan pengalaman.
- **evaluation return:** performa policy pada episode evaluasi terpisah.

Jangan melatih model menggunakan episode evaluasi.

11.3 Metrik Dasar

Metrik yang dapat digunakan:

Metrik	Pertanyaan yang dijawab
Mean episodic return	Berapa rata-rata hasil episode?
Median return	Apakah hasil tipikal berbeda dari rata-rata?
Success rate	Seberapa sering tujuan tercapai?
Episode length	Berapa langkah yang diperlukan?
Failure rate	Seberapa sering kondisi gagal terjadi?
Safety violation	Berapa kali batas keselamatan dilanggar?
Sample efficiency	Berapa banyak interaksi untuk mencapai performa tertentu?
Wall-clock time	Berapa lama training sebenarnya?

Reward tinggi tidak boleh menutupi safety violation.

11.4 Multiple Seeds

Jalankan eksperimen pada beberapa random seed, misalnya 5 sampai 10 seed untuk tugas pembelajaran. Laporan sebaiknya memuat:

- mean,

- standard deviation atau interval,
- kurva setiap seed bila memungkinkan,
- jumlah episode dan langkah,
- kriteria berhenti.

Untuk keputusan produksi, jumlah seed dan desain statistik perlu disesuaikan dengan risiko serta biaya.

11.5 Learning Curve

Kurva belajar biasanya menampilkan:

x-axis: environment steps atau episode
y-axis: evaluation return atau success rate

Gunakan moving average hanya untuk visualisasi. Simpan data mentah agar variasi tidak tersembunyi.

Perhatikan:

- kecepatan belajar,
- performa akhir,
- stabilitas,
- collapse setelah sempat baik,
- perbedaan antar-seed.

11.6 Baseline

Bandingkan dengan baseline sederhana:

- random policy,
- rule-based policy,
- greedy heuristic,
- algoritma tabular,
- versi tanpa komponen tertentu.

Model kompleks yang hanya sedikit lebih baik dari heuristic mungkin tidak sebanding dengan biaya dan risikonya.

11.7 Ablation Study

Ablation menghapus atau mengubah satu komponen untuk mengetahui kontribusinya.

Contoh DQN:

- tanpa target network,
- tanpa replay buffer,
- epsilon tetap vs decay,
- learning rate berbeda.

Ubah satu variabel utama pada satu waktu agar kesimpulan lebih jelas.

11.8 Reproducibility Checklist

Catat:

- versi Python dan library,

- environment dan versinya,
- seed,
- hardware,
- hyperparameter,
- preprocessing observation,
- desain reward,
- definisi terminated dan truncated,
- frekuensi evaluasi,
- checkpoint terbaik dan terakhir.

Checkpoint Bab 11

87. Mengapa satu seed tidak cukup?
88. Apa perbedaan training return dan evaluation return?
89. Mengapa baseline sederhana penting?
90. Apa tujuan ablation study?

Latihan Bab 11

Rancang tabel eksperimen untuk membandingkan tiga nilai epsilon decay. Sertakan:

- variabel yang dikontrol,
 - jumlah seed,
 - metrik utama,
 - metrik keselamatan,
 - aturan memilih konfigurasi terbaik.
-

Bab 12 - Mini Project: Q-Learning pada GridWorld

12.1 Tujuan Project

Peserta membangun agent yang belajar mencapai tujuan pada GridWorld sederhana sambil menghindari rintangan.

Capaian project:

- membuat environment sederhana,
- menerapkan Q-learning,
- menjalankan eksperimen beberapa seed,
- membandingkan hyperparameter,
- mengevaluasi return, success rate, dan panjang episode,
- mengaudit reward dan kegagalan agent.

12.2 Desain GridWorld

Gunakan grid 5 x 5.

```
S . . X .
. X . X .
. X . . .
. . . X .
X . . . G
```

Keterangan:

- **S**: start.
- **G**: goal.
- **X**: obstacle.
- **.**: sel yang dapat dilewati.

Action:

```
0 = atas
1 = kanan
2 = bawah
3 = kiri
```

Reward awal:

```
+10 mencapai goal
-5 menabrak obstacle atau keluar grid
-0.1 setiap langkah
```

Episode selesai saat goal tercapai atau langkah maksimum 100.

12.3 Implementasi Environment

```
class GridWorld:
    def __init__(self, size=5, max_steps=100):
        self.size = size
        self.max_steps = max_steps
        self.start = (0, 0)
        self.goal = (4, 4)
        self.obstacles = {(0, 3), (1, 1), (1, 3), (2, 1), (3, 3), (4, 0)}
```

```

        self.actions = [(-1, 0), (0, 1), (1, 0), (0, -1)]

    def reset(self):
        self.position = self.start
        self.steps = 0
        return self.position

    def step(self, action):
        self.steps += 1
        dr, dc = self.actions[action]
        candidate = (self.position[0] + dr, self.position[1] + dc)

        out = not (0 <= candidate[0] < self.size and 0 <= candidate[1] < self.size)
        blocked = candidate in self.obstacles

        if out or blocked:
            reward = -5.0
            next_position = self.position
        else:
            next_position = candidate
            reward = -0.1

        self.position = next_position
        terminated = self.position == self.goal
        if terminated:
            reward = 10.0

        truncated = self.steps >= self.max_steps and not terminated
        return self.position, reward, terminated, truncated

```

12.4 Implementasi Q-Learning

```

import random
from collections import defaultdict

def greedy_action(values):
    best = max(values)
    candidates = [i for i, value in enumerate(values) if value == best]
    return random.choice(candidates)

def train(seed=0, episodes=3000, alpha=0.1, gamma=0.95,
          epsilon_start=1.0, epsilon_min=0.05, epsilon_decay=0.995):
    random.seed(seed)
    env = GridWorld()
    q = defaultdict(lambda: [0.0] * 4)
    epsilon = epsilon_start
    history = []

    for episode in range(episodes):
        state = env.reset()
        total_reward = 0.0
        done = False

        while not done:
            if random.random() < epsilon:
                action = random.randrange(4)
            else:
                action = greedy_action(q[state])

            next_state, reward, terminated, truncated = env.step(action)
            done = terminated or truncated

            future = 0.0 if terminated else max(q[next_state])
            target = reward + gamma * future

```

```

q[state][action] += alpha * (target - q[state][action])

state = next_state
total_reward += reward

epsilon = max(epsilon_min, epsilon * epsilon_decay)
history.append(total_reward)

return q, history

```

12.5 Evaluasi Policy

```

def evaluate(q, episodes=100):
    env = GridWorld()
    returns = []
    successes = 0
    lengths = []

    for _ in range(episodes):
        state = env.reset()
        total = 0.0
        done = False

        while not done:
            action = greedy_action(q[state])
            state, reward, terminated, truncated = env.step(action)
            done = terminated or truncated
            total += reward

        returns.append(total)
        lengths.append(env.steps)
        successes += int(terminated)

    return {
        "mean_return": sum(returns) / len(returns),
        "success_rate": successes / episodes,
        "mean_length": sum(lengths) / len(lengths),
    }

```

12.6 Eksperimen Wajib

Jalankan minimal tiga konfigurasi:

Eksperimen	Alpha	Gamma	Epsilon decay
A	0.10	0.95	0.995
B	0.30	0.95	0.995
C	0.10	0.99	0.999

Untuk setiap konfigurasi:

- gunakan minimal lima seed,
- catat mean return,
- catat success rate,
- catat panjang episode,
- tampilkan learning curve,
- jelaskan variasi antar-seed.

12.7 Audit Reward Project

Jawab pertanyaan berikut:

91. Apakah penalti -5 membuat agent terlalu takut mengeksplorasi?
92. Apakah biaya langkah -0.1 cukup mendorong jalur pendek?
93. Apa yang terjadi jika reward goal dinaikkan menjadi $+100$?
94. Apakah agent dapat berputar tanpa akhir jika max steps dihapus?
95. Apakah obstacle seharusnya langsung mengakhiri episode?

12.8 Deliverable

Kumpulkan:

96. Notebook atau file Python.
97. README singkat.
98. Tabel konfigurasi eksperimen.
99. Grafik learning curve.
100. Ringkasan hasil maksimal dua halaman.
101. Analisis minimal tiga kegagalan agent.
102. Audit reward dan rekomendasi perbaikan.

12.9 Rubrik Penilaian

Komponen	Bobot
Kebenaran implementasi environment dan Q-learning	25%
Desain eksperimen dan penggunaan beberapa seed	20%
Evaluasi dan visualisasi	20%
Analisis reward, risiko, dan kegagalan	20%
Kerapian kode dan komunikasi hasil	15%

Bab 13 - Kuis Akhir

Pilih jawaban yang paling tepat.

Soal

103. Komponen yang memilih action dalam RL adalah ...
 - A. Reward
 - B. Agent
 - C. Dataset
 - D. Label
104. Tujuan umum RL adalah ...
 - A. Memaksimalkan satu reward langsung saja
 - B. Meminimalkan ukuran state
 - C. Memaksimalkan expected return jangka panjang
 - D. Menghapus kebutuhan evaluasi
105. State yang baik seharusnya ...
 - A. Selalu berupa gambar
 - B. Merangkum informasi relevan untuk keputusan berikutnya
 - C. Tidak berubah sepanjang episode
 - D. Hanya berisi reward
106. Discount factor mendekati nol membuat agent ...
 - A. Lebih fokus pada reward dekat
 - B. Selalu memilih action acak
 - C. Tidak dapat belajar
 - D. Mengabaikan reward langsung
107. $V(s)$ memperkirakan ...
 - A. Probabilitas state tidak pernah muncul
 - B. Return dari state ketika mengikuti policy
 - C. Jumlah action yang tersedia
 - D. Learning rate optimal
108. $Q(s, a)$ memperkirakan ...
 - A. Return setelah mengambil action a pada state s
 - B. Ukuran replay buffer
 - C. Jumlah episode
 - D. Nilai reward terakhir saja
109. Intuisi Bellman adalah ...
 - A. Nilai sekarang terdiri dari reward langsung dan nilai masa depan
 - B. Semua state harus memiliki nilai sama
 - C. Reward tidak diperlukan
 - D. Policy tidak boleh berubah
110. Exploration diperlukan agar ...
 - A. Agent tidak pernah menggunakan pengetahuan lama
 - B. Agent dapat menemukan action yang belum cukup dicoba
 - C. Episode selalu lebih panjang
 - D. Reward menjadi nol
111. Pada epsilon-greedy, epsilon menyatakan ...

- A. Probabilitas eksplorasi
 - B. Discount factor
 - C. Learning rate
 - D. Ukuran state
112. Monte Carlo memperbarui nilai ...
- A. Sebelum menerima reward
 - B. Setelah memperoleh return episode
 - C. Tanpa pengalaman
 - D. Hanya pada supervised dataset
113. Temporal-Difference disebut bootstrapping karena ...
- A. Menggunakan estimasi nilai berikutnya untuk memperbarui nilai sekarang
 - B. Selalu memakai neural network
 - C. Tidak memiliki reward
 - D. Menghapus state terminal
114. SARSA adalah on-policy karena targetnya memakai ...
- A. Action acak dari policy lain
 - B. Action berikutnya yang benar-benar dipilih policy saat ini
 - C. Hanya reward maksimum global
 - D. Dataset tetap
115. Target Q-learning memakai ...
- A. Nilai minimum action berikutnya
 - B. Rata-rata semua reward historis
 - C. Maksimum Q pada state berikutnya
 - D. Tidak ada nilai masa depan
116. Replay buffer pada DQN berguna untuk ...
- A. Menyimpan model final saja
 - B. Menggunakan ulang dan mengacak pengalaman training
 - C. Menghapus kebutuhan target network
 - D. Menjamin keselamatan
117. Target network membantu ...
- A. Membuat target belajar lebih stabil
 - B. Menambah jumlah action
 - C. Mengubah state menjadi reward
 - D. Menghapus exploration
118. Policy gradient mengoptimalkan ...
- A. Policy secara langsung
 - B. Ukuran replay buffer saja
 - C. Jumlah state
 - D. Label klasifikasi
119. Critic pada actor-critic berfungsi untuk ...
- A. Memilih warna visualisasi
 - B. Menilai state atau action dan memberi sinyal belajar
 - C. Menghapus reward
 - D. Menentukan hardware
120. PPO membatasi perubahan policy agar ...
- A. Update tidak terlalu ekstrem
 - B. Semua action memiliki probabilitas nol

- C. Policy tidak pernah belajar
 - D. Reward selalu positif
121. Evaluasi RL sebaiknya ...
- A. Hanya memakai satu seed terbaik
 - B. Memisahkan evaluasi dari training dan memakai beberapa seed
 - C. Hanya melihat loss network
 - D. Mengabaikan safety violation
122. Reward hacking berarti ...
- A. Agent mendapat reward tinggi melalui perilaku yang tidak sesuai tujuan sebenarnya
 - B. Programmer menghapus reward
 - C. Reward selalu negatif
 - D. Agent menggunakan epsilon-greedy

Kunci Jawaban dan Pembahasan Ringkas

123. **B** - Agent memilih action.
124. **C** - Fokusnya expected return jangka panjang.
125. **B** - State perlu memuat informasi relevan untuk konsekuensi berikutnya.
126. **A** - Reward jauh semakin kecil bobotnya.
127. **B** - V mengukur nilai state menurut policy.
128. **A** - Q mengukur nilai pasangan state-action.
129. **A** - Nilai dipecah menjadi reward langsung dan nilai masa depan.
130. **B** - Eksplorasi menghasilkan informasi baru.
131. **A** - Epsilon adalah peluang memilih action eksploratif.
132. **B** - Monte Carlo memakai return setelah episode.
133. **A** - TD menggunakan estimasi nilai berikutnya.
134. **B** - SARSA mengikuti action aktual dari policy saat ini.
135. **C** - Q-learning memakai action terbaik menurut estimasi berikutnya.
136. **B** - Replay meningkatkan reuse data dan mengurangi korelasi sampel.
137. **A** - Target network mengurangi perubahan target yang terlalu cepat.
138. **A** - Policy gradient memperbarui parameter policy langsung.
139. **B** - Critic menghasilkan estimasi value atau advantage.
140. **A** - Clipping mencegah policy berubah terlalu jauh dalam satu update.
141. **B** - Evaluasi harus terpisah dan mengukur variasi antar-seed.
142. **A** - Agent mengeksploitasi celah reward tanpa memenuhi maksud manusia.

Diskusi dan Refleksi

Gunakan pertanyaan berikut untuk diskusi kelompok:

143. Dalam konteks pendidikan, apa bahaya jika reward hanya berdasarkan waktu penggunaan aplikasi?
144. Apakah agent yang memperoleh reward tertinggi selalu menjadi agent terbaik? Jelaskan.
145. Bagaimana cara membatasi eksplorasi pada sistem yang melibatkan manusia?
146. Kapan heuristic sederhana lebih tepat daripada RL?
147. Bagaimana memastikan kelompok pengguna yang jarang muncul tidak dirugikan oleh policy?
148. Apa perbedaan antara simulator yang nyaman untuk training dan simulator yang valid untuk deployment?

149. Jika dua algoritma memiliki return sama tetapi satu lebih stabil, mana yang dipilih dan mengapa?
150. Bagaimana menjelaskan keputusan agent RL kepada pemangku kepentingan non-teknis?

Glosarium

Istilah	Penjelasan
Action	Keputusan yang dipilih agent
Actor	Komponen yang menghasilkan policy atau action
Advantage	Seberapa baik suatu action dibanding ekspektasi pada state
Agent	Sistem yang belajar dan mengambil keputusan
Bellman equation	Hubungan rekursif antara nilai sekarang, reward, dan nilai masa depan
Bootstrapping	Memperbarui estimasi menggunakan estimasi lain
Critic	Komponen yang memperkirakan value atau advantage
Discount factor	Bobot untuk reward masa depan
DQN	Neural network untuk memperkirakan Q-value pada action diskret
Environment	Dunia atau sistem tempat agent berinteraksi
Episode	Satu rangkaian interaksi dari awal hingga akhir
Exploration	Mencoba action untuk memperoleh informasi
Exploitation	Memilih action yang saat ini dianggap terbaik
MDP	Model matematis untuk keputusan berurutan dengan state, action, transisi, reward, dan discount
Observation	Informasi yang diterima agent tentang environment
Off-policy	Belajar tentang policy yang berbeda dari policy pengumpulan action
On-policy	Belajar tentang policy yang sedang digunakan
Policy	Strategi memilih action dari state
Q-value	Perkiraan return untuk pasangan state dan action
Replay buffer	Penyimpanan pengalaman untuk digunakan ulang dalam training
Return	Akumulasi reward yang telah didiskon
Reward	Sinyal numerik dari environment
Reward hacking	Perilaku mengejar reward dengan cara yang tidak sesuai tujuan sebenarnya
State	Representasi keadaan yang relevan bagi keputusan

Istilah	Penjelasan
TD error	Selisih antara target TD dan estimasi saat ini
Value function	Perkiraan kualitas jangka panjang suatu state atau state-action

Ringkasan Cepat

- RL mempelajari keputusan melalui interaksi agent dan environment.
- Agent mengoptimalkan return jangka panjang, bukan reward satu langkah.
- MDP membantu memodelkan state, action, transisi, reward, dan discount.
- Policy menentukan action; value function menilai masa depan.
- Bellman memecah nilai menjadi reward langsung dan nilai berikutnya.
- Exploration diperlukan untuk memperoleh informasi baru.
- Monte Carlo belajar dari episode lengkap; TD belajar setiap langkah.
- SARSA bersifat on-policy; Q-learning bersifat off-policy.
- DQN memakai neural network, replay buffer, dan target network.
- Policy gradient mengoptimalkan policy langsung; actor-critic memadukan actor dan critic.
- PPO membatasi perubahan policy agar tidak terlalu ekstrem.
- Reward tinggi tidak cukup: evaluasi harus mencakup keselamatan, stabilitas, dan dampak.
- Eksperimen perlu beberapa seed, baseline, metrik terpisah, dan dokumentasi lengkap.

Referensi Belajar

151. Richard S. Sutton dan Andrew G. Barto. **Reinforcement Learning: An Introduction**, edisi kedua.
152. Christopher J. C. H. Watkins dan Peter Dayan. **Q-learning**.
153. Volodymyr Mnih dkk. **Human-level control through deep reinforcement learning**.
154. John Schulman dkk. **Proximal Policy Optimization Algorithms**.
155. Gymnasium Documentation - API environment untuk reinforcement learning.
156. OpenAI Spinning Up - pengantar algoritma deep reinforcement learning.
157. David Silver - Reinforcement Learning lecture materials.

Catatan Penutup

Reinforcement learning memerlukan lebih dari memilih algoritma. Kualitas state, reward, simulator, batas keselamatan, dan evaluasi menentukan apakah agent benar-benar menyelesaikan masalah. Mulailah dari environment kecil, gunakan baseline sederhana, dokumentasikan eksperimen, dan selalu periksa perilaku agent di luar angka reward.